



An ontology for reasoning over engineering textual data stored in FMEA spreadsheet tables[☆]

Melinda Hodkiewicz^{a,*}, Johan W. Klüwer^b, Caitlin Woods^c, Thomas Smoker^c, Emily Low^c

^a School of Mechanical Engineering, University of Western Australia, Perth, WA, Australia

^b DNV, Oslo, Norway

^c School of Computer Science & Software Engineering, University of Western Australia, Perth, WA, Australia

ARTICLE INFO

Article history:

Received 4 March 2021

Received in revised form 3 May 2021

Accepted 27 May 2021

Keywords:

Reasoning

Maintenance

FMEA

Artificial intelligence

Ontology

Failure mode

ABSTRACT

Much textual engineering knowledge is captured in tables, particularly in spreadsheets and in documents such as equipment manuals. To leverage the benefits of artificial intelligence, industry must find ways to extract the data and relationships captured in these tables. This paper demonstrates the application of an ontological approach to make the classes and relations held in spreadsheet tables explicit. Ontologies offer a pathway because they use formal descriptions to define machine-interpretable definitions of shared concepts and relations between concepts. We illustrate this with two case studies on a failure modes and effects analysis (FMEA) table. Our examples demonstrate how the relationship between rows and columns in a table can be represented in logic for FMEA entries, thereby allowing the same ontology to ingest instance data from the IEC 60812:2006 FMEA Standard and a real industrial FMEA. We give relationships in the FMEA and asset hierarchy spreadsheets an explicit representation, so that OWL-DL reasoning can infer final failure effects at the system level from component failures. The prototype ontologies described in this paper are modular and aligned to a top level ontology, and hence can be applied to other use cases. Our contribution is showing that engineers can make data captured in commonly used spreadsheet tables machine readable using a FMEA ontology.

© 2021 Published by Elsevier B.V.

1. Introduction

In a data-driven maintenance future, the engineering community will have to decide how to handle the vast knowledge base held in spreadsheet tables. Examples include the knowledge captured in failure modes and effects analysis (FMEA) (Hodkiewicz et al., 2020), reliability-centered maintenance strategy development, and failure investigations. This information is central to making use of artificial intelligence capabilities like automated reasoning, which

enables computers to perform tedious and repetitive work currently done by technical people, and hence improve productivity and quality control.

We focus on the FMEA process, which is a risk identification and assessment process widely used in systems engineering and asset management. The engineering text data captured in FMEA tables is usually semi-structured or unstructured. Specific naming of the column headings for the different fields is often ad hoc and this limits integration with other systems. Extraction and alignment of data from tables created by different groups is therefore a manual process and this limits re-use of the FMEA analysis (Rehman and Kifor, 2016) and use of automated reasoning to improve the quality of the process. The use of ontologies could potentially assist in streamlining this process and reducing the manual effort. An ontology is an explicit and formal specification of a shared conceptualization (Studer et al., 1998). It defines the concepts, properties, relationships, functions, axioms, and constraints for a field of interest – in our case, FMEA.

Tables are efficient for editing and storing data, but when they are used in schema-less formats, like spreadsheets, a user-selected column heading has no semantics associated with it. Hence, it can-

[☆] This paper is an extended version of a paper presented at the 4th IFAC A-MEST workshop in 2020. This work would not have been possible without funding from the BHP Fellowship for Engineering for Remote Operations – supporting community projects in areas in which BHP operates. Hodkiewicz is also part of the Australian Research Council Centre for Transforming Maintenance through Data Science (Industrial Transformation Research Program Grant No. IC180100030).

* Corresponding author.

E-mail addresses: melinda.hodkiewicz@uwa.edu.au (M. Hodkiewicz), Johan.Wilhelm.Kluewer@dnv.com (J.W. Klüwer), caitlin.woods@uwa.edu.au (C. Woods), thomas.smoker@uwa.edu.au (T. Smoker), emily.low@uwa.edu.au (E. Low).

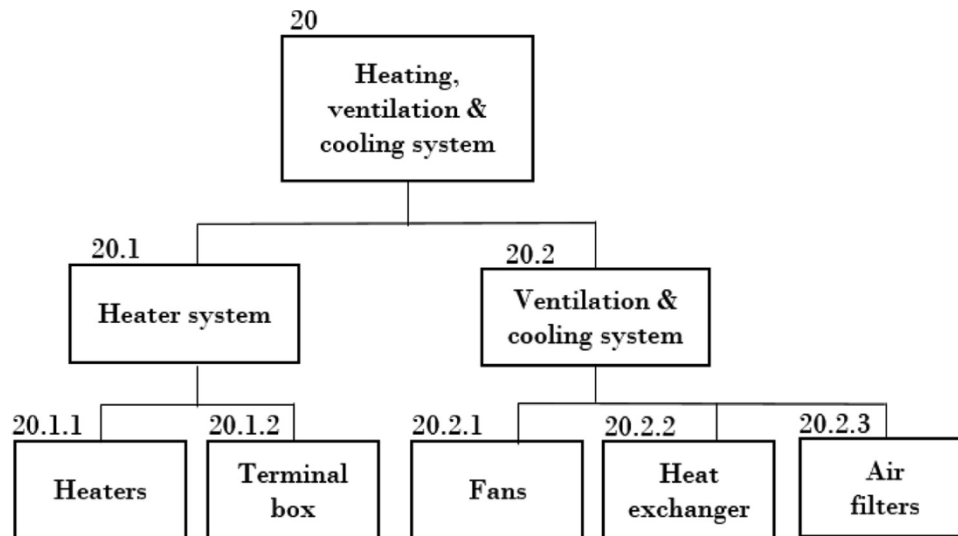


Fig. 1. Part/whole structure of the heating, ventilation and cooling system, extract from ISO 60812:2006.

not be interpreted as-is using a reasoner. This issue is magnified as relationships between columns are undefined. However, FMEA is an engineering process described in engineering standards. As a result, there are widely-accepted ways to present the data in the table structure and to use common concepts. This creates an implicit schema for the table. An ontology (Staab and Studer, 2010) can be used to capture the logic of these implicit relationships between columns in tables explicitly.

In the case of FMEA, the industry has the IEC 60812:2006 FMEA Standard (IEC, 2006a). The Standard makes use of a heater system and its components for illustrative purposes. The engineering reader knows that a heater system cannot function without a heater as the heater is a component of the heater system in the asset hierarchy described in the Standard. This relationship, which is both physical and functional, can be explicitly coded as an axiom in an ontology, and hence become interpretable by logical reasoner software (such as, for OWL, Pellet or HermiT). For example, heater system and heater can be related by a *hasFunctionalPart* relationship. Use of a reasoner allows checks to be made for consistency within a table and across other similar tables. This type of automated checking can be used to identify quality control issues in the FMEA process.

Accordingly, we propose the use of ontologies as a way to represent the data currently stored in FMEA spreadsheet tables and perform quality checks. In this work, we discuss both the ontology's development and data ingestion process. This is because the work is aimed at being able to deal with data sets that are stored in tables by industry engineers. The representation of classes and relationships in the ontology influences the selection and use of the data ingestion approach and hence we describe both ontology development and data management. The aim is to demonstrate an end to end process, starting with industry data for a FMEA and associated asset hierarchy, producing a set of modular, inter-operable ontologies aligned to a top level ontology, and demonstrating reasoning to enable quality control checks for industry FMEA users.

The paper is organized as follows. First, we describe the development use case. This description includes some background on FMEA. We have selected FMEA as it is a widely used engineering process whose outputs are commonly stored in tables. Section 3 describes a data set and in Section 4 we describe our ontology design approach and ontology modularisation choices. We use the development use case to inform design and test reasoning. Section 5 describes the ability to infer failure effects at the system level from

component failures, as cause and effect relationships are propagated using OWL-DL reasoning. An ontology was developed in order to demonstrate this reasoning. Section 6 tests the ontology with a new (test) use case in which tabular data, provided by an industry partner for the FMEA on a pressure vessel, is ingested into the ontology and competency questions that demonstrate reasoning are addressed. This allows us to check the ability of the ontology to be applied to a different use case. Finally, in Section 7 we discuss the contribution of this work to the engineering and ontology communities and suggest next steps. A GitHub repository containing the ontologies presented in this paper is available at https://github.com/uwasystemhealth/Paper_Archive_CII_FMEA_Ontology.

2. Use case

Failure modes and effects analysis is a widely used process in systems engineering and for risk assessment in engineering and business. The application of FMEA is preceded by a hierarchical decomposition of the asset system into its functional sub-systems. For demonstration purposes, we use a use case from the appendix of the IEC Standard for Analysis Techniques for System Reliability-Procedure for Failure Mode and Effects Analysis Standard, IEC 60812:2006 (IEC, 2006b). The example in the IEC Standard was selected because it is publicly available, has been peer-reviewed, and is simple enough to illustrate the points we are making about the challenges with reasoning over textual data commonly stored in spreadsheet tables. In the testing section, we apply the same pipeline to ingest data from tables storing FMEA data for an industrial pressure vessel and perform reasoning over this real, and more complex, data.

The hierarchical structure of the asset breakdown in the IEC 60812 example is shown in Fig. 1. There are two subsystems, the Heater System [20.1] and the Ventilation and cooling system [20.2]. The Heater system [20.1], the focus in this paper, has two sub-systems, Heaters [20.1.1] and Terminal box [20.1.2]. This hierarchy may also be represented in a spreadsheet like Table 1. Note that this Heating, ventilation and cooling subsystem is itself a subsystem in a deeper hierarchy of, for example, a compressor, engine or HVAC system. Each component has relationships to other components, and may have multiple failure modes. As a result, there can be hundreds of items in a typical FMEA risk assessment.

Table 1

Example of a spreadsheet for selected instances from the asset hierarchy in Fig. 1.

isFunctionalPartOf	Heating, ventilation & cooling system	Heater system	Ventilation & cooling system
Heater system	X		
Ventilation & cooling system	X		
Heater		X	
Terminal box		X	
Fans & fan hubs			X
Heat exchanger			X
Air filters			X

Table 2

Extract of spreadsheet table for FMEA example for the Heater system (IEC, 2006b).

Ref.	Component	Function	Failure mode	Failure effect
20.1.1	Heaters	To heat up unit	(a) overcurrent (b) short circuit (c) earth fault	Loss of all heating. Loss of all heating. Loss of all heating
20.1.2	Terminal box	Connect supply to heaters	(a) overcurrent (b) short circuit (c) cable failure	Loss or reduction of heating Loss of all heating Loss or reduction of heating

3. Data

Analysis to identify failure modes and associated failure effects starts at the lowest functional level of the asset hierarchy. The effect of a failure mode at a low level can become a failure cause of a higher level component, and these effects are propagated up through the system hierarchy until the final failure effect is identified.

The outputs of the FMEA assessment are generally stored in spreadsheets. Table 2 shows entries for a FMEA for the Heaters and Heater Terminal box. Each component has a function, failure mode and associated failure effect. The propagation of a failure through the system depends on the dependencies between components in the system and their redundancy. In this example the failure of a Heater [20.1.1] causes failure of the Heating System [20.1], which in turn causes failure of Heating, ventilation and cooling system [20]. The reasoning process across two separate spreadsheets, asset hierarchy and FMEA, is what we wish to support. It is important to note that the tabular data cannot be used in automated reasoning unless processed via something like an ontology.

Some challenges with using spreadsheets to store FMEA data are, (1) it is difficult to ensure that terms and relationships are used consistently, particularly when the tables are large, (2) the language used is often specific to those involved in a particular FMEA development exercise, and (3) a spreadsheet typically has no explicit semantics, all of which makes it difficult to find, share or reuse the knowledge acquired during the analysis. Making the information contained in FMEA spreadsheet tables machine readable and explicit would address these issues.

4. Ontology design

While our focus is on digitalization and reasoning over engineering spreadsheets, our use case is FMEA. So, references to previous attempts to develop FMEA ontologies are in order. Rehman et al. identify classes and relationships for FMEA concepts with the aim of being able to extract, store and retrieve information (Rehman and Kifor, 2016). While their work lacks a real use case it nevertheless provides motivation for this work as they identify their next steps as “auto-population of a knowledge base from CSV format FMEA worksheets”. FMEA has been of interest to the ontology community since the work of (Dittman et al., 2004) in the early 2000s which established the potential for the transitive properties of an ontology to propagate failures through the asset structure. How-

ever much of the work is on conceptual developments in general (Ebrahimipour et al., 2010), or for specific applications (Laaroussi et al., 2007; Molhanec et al., 2011). Instance data or a substantial demonstration of the use of reasoning is rare. There are papers describing software applications that include a FMEA ontology base (e.g. (Xiuxu and Yuming, 2012)) but from the details given it is difficult to assess their effectiveness or use. In summary, while previous work makes the case for why there should be FMEA ontologies, there is a lack of examples with real use cases, reasoning that leverages the properties and axioms in the ontology, and examples that deal with data from more than one table.

The principles guiding our design are (1) application of a design pattern for disposition and function aligned to a top level ontology, (2) using asset hierarchy and standard descriptors from engineering standards, and (3) supporting modular ontology development principles by keeping the FMEA ontology separate from the asset hierarchy. In developing this ontology we are guided by the process described in (Uschold and Gruninger, 1996).

We propose the use of a hierarchy of four ontologies (Fig. 2). From left to right, these are ISO 15926-14 (ISO 15926-14, 2019), an OWL-DL top-level ontology (ISO), a Functional System ontology (FSO), a FMEA ontology (FMO) and an Asset System ontology (ASO). Here, FSO provides classes and relations to represent a heater system and its components; whereas the FMO extends this to failure effects and failure mode observations. The ASO contains the systems and components of a particular asset, in a functional breakdown structure, with data as in Table 2.

Currently, determining a final failure effect requires engineers to scroll through the FMEA spreadsheet and cross-reference with the asset hierarchy. This manual approach is time consuming and prone to human error. We wish to use the inferential power of the ontology to identify final failure effect without the need for human intervention.

4.1. The Functional System ontology (FSO)

The FSO introduces four classes: Process, Component, MechanicalSystem and AncillarySystem. These allow us to represent functional structure and relationships of an asset. See Fig. 3, with ISO 15926 classes in blue and use case classes in white. Relations are from ISO 15926.



Fig. 2. The structure of the ontologies and their imported ontologies.

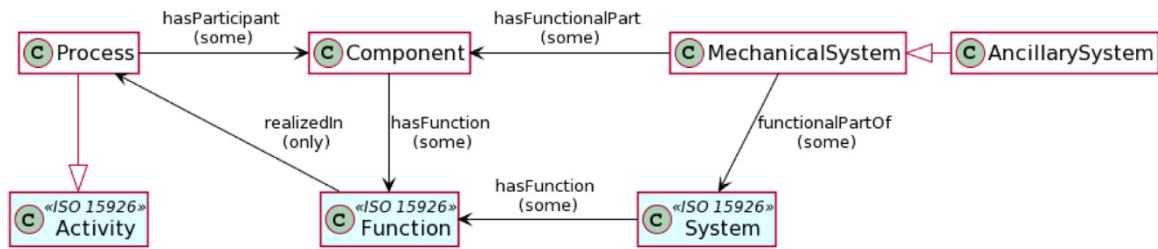


Fig. 3. The functional system ontology. 'C' represents classes and white-headed arrows are subclass relations. (For interpretation of the references to color in the text, the reader is referred to the web version of this article.)

Table 3

Description of the class terms in the FMEA ontology.

Term	Description	Formal definition
FailureEffect	The consequence of a 'FailureMode' in terms of the operation, function or status of the item (IEC 60812:2008).	An FSO: process that results from an FMO: functional failure.
FinalFailureEffect	The final consequence on the system (IEC 60812:2008).	The terminal FSO: process from the set of FSO: processes that result from an FMO: functional failure.
FailureMode Observation	The effect by which a failure is observed on the failed item (ISO 14224:2006).	An ISO: information object that represents an FMO: functional failure.
Component Function	Subclass of 'Function'.	An ISO: function which is realized only in some FSO: process.
PrimaryFunction	The component's primary function, a subclass of 'ComponentFunction'.	The FMO: component function that is central to an FSO: component's participation in an FSO: process.
Malfunction	The disposition of an item to fail to perform a required function.	An ISO: disposition that is only realized in an FMO: failure event.
ObjectIn FaultState	The state of an item resulting from a 'Malfunction'.	An ISO: functional object that has some realized FMO: malfunction.
FailureEvent	A process or activity which results in functional failure of an item. Equivalent to (hasParticipant some Component) and (resultsIn some FunctionalFailure).	An FSO: process that results in an FMO: functional failure.
Functional Failure	The termination of ability of an item to perform a required function (IEC 60812:2008). Equivalent to (representedBy some FailureModeObservation) and (resultsIn some FailureEffect).	An FSO: process which is the result of an FMO: failure event and results in an FMO: failure effect. It is represented by some FMO: failure mode observation.

4.2. The FMEA ontology (FMO)

For the FMO, the challenge is to provide definitions for the vocabulary used in the FMEA process. We wish to capture the engineering intent as a special-purpose application of the generic pattern prescribed for the top level ontology. We propose nine classes: FailureEffect, FinalFailureEffect, FailureModeObservation, ComponentFunction, PrimaryFunction, Malfunction, ObjectInFaultState, FailureEvent, and FunctionalFailure (Table 3).

Note that our ontology contains more classes than are listed in Table 2, as extra classes are needed to support ontology reasoning. They would not be required if we were just capturing the data in the table as a set of triples in a knowledge graph. The classes we have used, with definitions and relationships to the ISO, FMO and FSO ontologies, are shown in Table 3. For example, consider the triple (20.1, hasComponent, Heaters) in Table 1. The classes Malfunction and ObjectInFaultState are crucial. Malfunctions are analogous to functions, but represent known ways that the entity can engage in processes that are contrary to intended purpose. When an individual has a 'Malfunction', we mean it has the potential to participate in a 'FunctionalFailure' process (in detail: Malfunction is a subclass of Disposition, and is realizedIn only FailureEvent). Like intended func-

tions, a (disposition to) malfunction may or may not be realized. If it is, the system or component becomes an ObjectInFaultState, secured by the following axiom.

```
hasMalfunction some (realizedIn some owl:Thing)
SubClassOf ObjectInFaultState
```

This is illustrated in Fig. 4, using data extracted and mapped from Table 2. In this graphical representation, boxes with a green 'c' are classes, and those with a pink 'i' are individuals. Red arrows with open arrowheads represent class-subclass relationships, blue arrows with a $\in$ symbol represent membership in a class, and black arrows represent object-to-object relationships.

Propagation of fault state is prescribed by the axiom beneath Fig. 4. We make the simplifying assumption that any system that has a sub-system or component which is in a fault state is itself in a fault state. This pattern of inference is illustrated in Fig. 5.

```
hasfunctionalPart some ObjectInFaultState
SubClassOf ObjectInFaultState
```

4.3. Asset System ontology (ASO)

The ASO extends the FMEA ontology with a representation of the particular asset being analysed. It introduces classes, for our case,

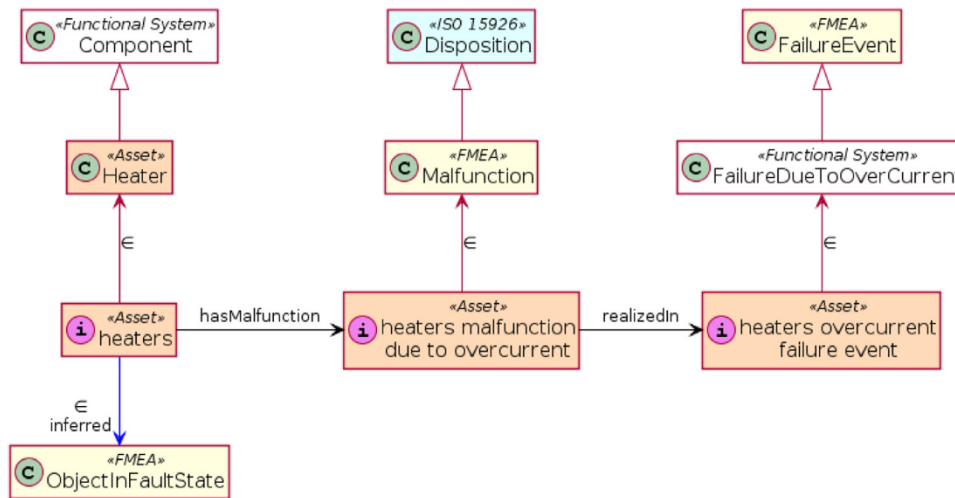


Fig. 4. The *heaters* object has an identified disposition to malfunction, and the malfunction is realized. 'C' represents classes, 'i' represents individuals, white-headed arrows are subclass relations and € arrows are membership relations. (For interpretation of the references to color in the text, the reader is referred to the web version of this article.)

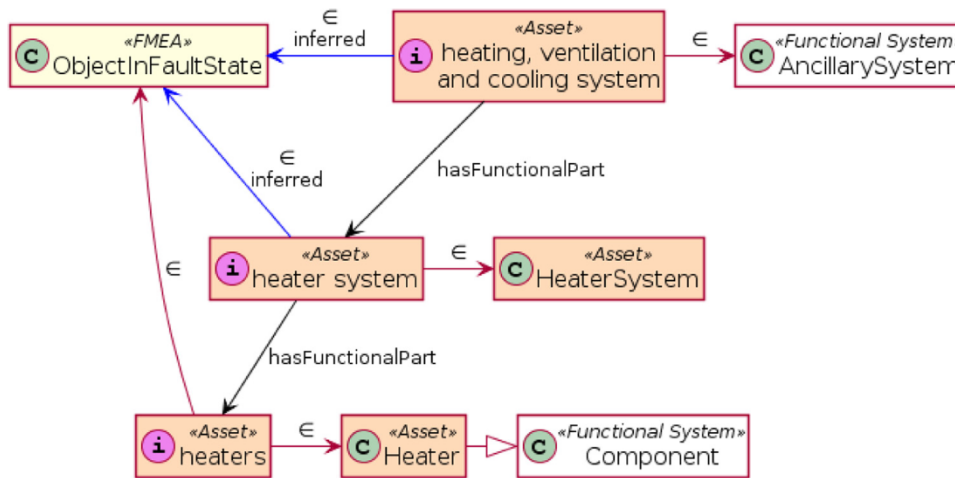


Fig. 5. Using inference to propagate a fault state from a realized malfunction.

to represent a heater system and component types. Each part of the system is represented as an individual instance of one of these classes.

For this use case, the system classes represent the top level items: Heater and Ventilation systems. The use case data produces individual instances. For example, the item '[20.1.1] Heaters' is a member of the class 'Heater', and a functional part of '[20.1] Heater system', which again is a member of the class 'Heating System'.

Two types of heater failure effects are listed in Table 2: 'reduced heating' and 'loss of all heating'. We introduce the class 'Heating Process', with two subclasses, 'Heating attempt and no heat' and 'Heating attempt and partial heat', to represent the failure effects – unwanted processes in which the malfunctions are realized.

An overview of the combined ontology is shown in Fig. 6. The colours show ISO in blue, FSO in white, and FMO in yellow. Collectively they provide a vocabulary for representing, in a machine readable way, the data captured in the FMEA and asset hierarchy tables.

5. Inference on the development use case

Here, inference aims to (1) ensure consistency in the use of terms and relationships in the FMEA and asset hierarchy spreadsheet tables, and (2) propagate the effect of the component failure

through the system as a check on the engineer's reasoning. For (1), the ontology will not "compile" if there are logical inconsistencies in the way classes and relationships have been used. To illustrate (2), our FMEA analysis tells us that *heaters* has three failure modes: overcurrent, short circuit and earth fault. We represent them as individuals in the ontology, for example 'heaters malfunction due to overcurrent'. To investigate how this failure mode will play out, we add a process individual, 'heaters overcurrent failure event', which allows us to represent a case in which the failure mode (malfunction) is realized.

The consequent failure effect will be propagated in a sequence of reasoning steps, using any OWL-DL reasoner. The details can be conveniently shown with the Explanation facility of the Protégé ontology workbench (Horridge et al., 2008; Horridge, 2011). It will be inferred that the *heaters* individual is an *ObjectInFaultState* (Fig. 7). The fault state of the heaters is propagated through to the Heater System and further to the Ancillary System, the top level of the asset hierarchy (Fig. 8).

6. Test use case – pressure vessel FMEA

Data (including a spreadsheet capturing the results of a FMEA analysis) and figures for functional and physical arrangements for components in a pressure vessel (PV) were obtained from an

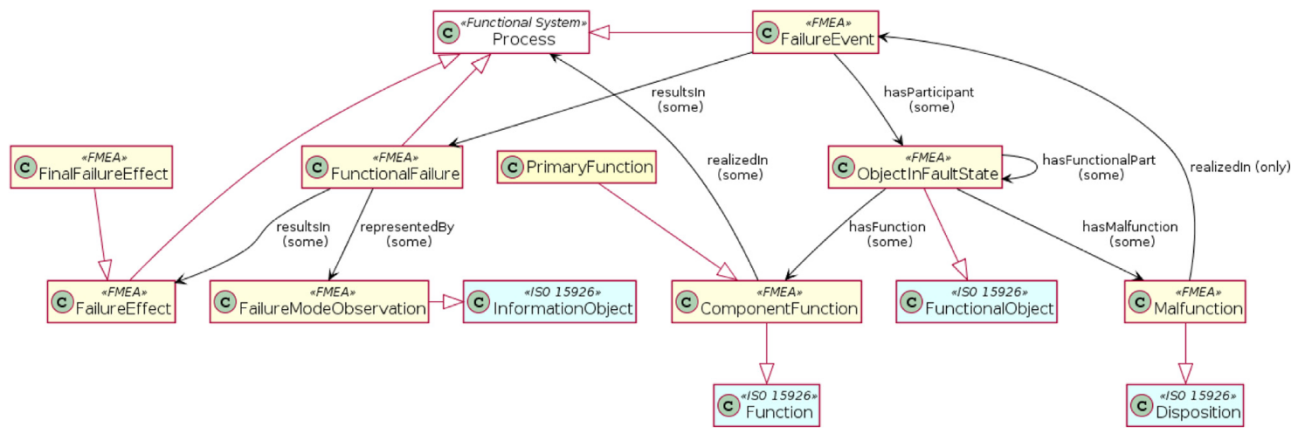


Fig. 6. Schematic illustrating the classes in the combined asset ontology showing relations to the modular ontologies. (For interpretation of the references to color in the text, the reader is referred to the web version of this article.)

Explanation for: heaters Type 'Object in Fault State'

heaters hasMalfunction 'heaters malfunction due to over current'	?
hasMalfunction some (realizedIn some owl:Thing) SubClassOf 'Object in Fault State'	?
'heaters malfunction due to over current' realizedIn 'heaters overcurrent failure event'	?

Fig. 7. Inference that *heaters* is an 'ObjectInFaultState' due to overcurrent.

Explanation for: 'Enclosure heating, ventilation and cooling system' Type 'Object in Fault State'

'Enclosure heating, ventilation and cooling system' hasFunctionalPart 'Heater system'	?
'Heater system' hasFunctionalPart heaters	?
heaters hasMalfunction 'heaters malfunction due to over current'	?
'heaters malfunction due to over current' realizedIn 'heaters overcurrent failure event'	?
hasMalfunction some (realizedIn some owl:Thing) SubClassOf 'Object in Fault State'	?
hasFunctionalPart some 'Object in Fault State' SubClassOf 'Object in Fault State'	?

Fig. 8. Inference that the Ancillary System is an 'ObjectInFaultState'.

Table 4

Extract of pressure vessel FMEA from industrial partner.

FMEA.ID	Functional location	ISO14224 Failure mode code	ISO14224 Failure mode description
FMEA.1000.10.10-RV001-001	1000.10.10-RV001-AG01-DRIV-GBOX	VIB	Vibration
FMEA.1000.10.10-RV001-002	1000.10.10-RV001-AG01-DRIV-GBOX	STD	Structural deficiency
FMEA.1000.10.10-RV001-003	1000.10.10-RV001-AG01-DRIV-GBOX	VIB	Vibration
FMEA.1000.10.10-RV001-004	1000.10.10-RV001-AG01-DRIV-GBOX	VIB	Vibration
ISO14224 failure mechanism	Function	Hidden failure	Failure effect(s)
Fatigue	Speed Reduction & Rotational Torque for Agitator assembly	Evident	Non Functional (gears damage – agitator out service – pressure vessel out of service)
Erosion	Speed Reduction & Rotational Torque for Agitator assembly	Evident	Non Functional (gears damage – agitator out service – pressure vessel out of service)
Breakage	Speed Reduction & Rotational Torque for Agitator assembly	Evident	Non Functional (gears damage – agitator out service – pressure vessel out of service)
Deformation	Speed Reduction & Rotational Torque for Agitator assembly	Evident	Non Functional (gears damage – agitator out service – pressure vessel out of service)

industrial partner. The FMEA was done to develop a maintenance strategy. An extract of the pressure vessel FMEA table is shown in Table 4 and elements in the pressure vessel system are shown in Fig. 9. This extract focuses on the function, failure modes, mechanisms and effects for the gearbox (AG01-DRIV-GBOX) on providing

torque for the drive unit of the agitator used to keep materials inside the pressure vessel in suspension, described here as “speed reduction and rotational torque for agitator assembly”. There are two failure modes (a) vibration (VIB), and (b) structural damage associated with material issues such as wear and corrosion (STD). These

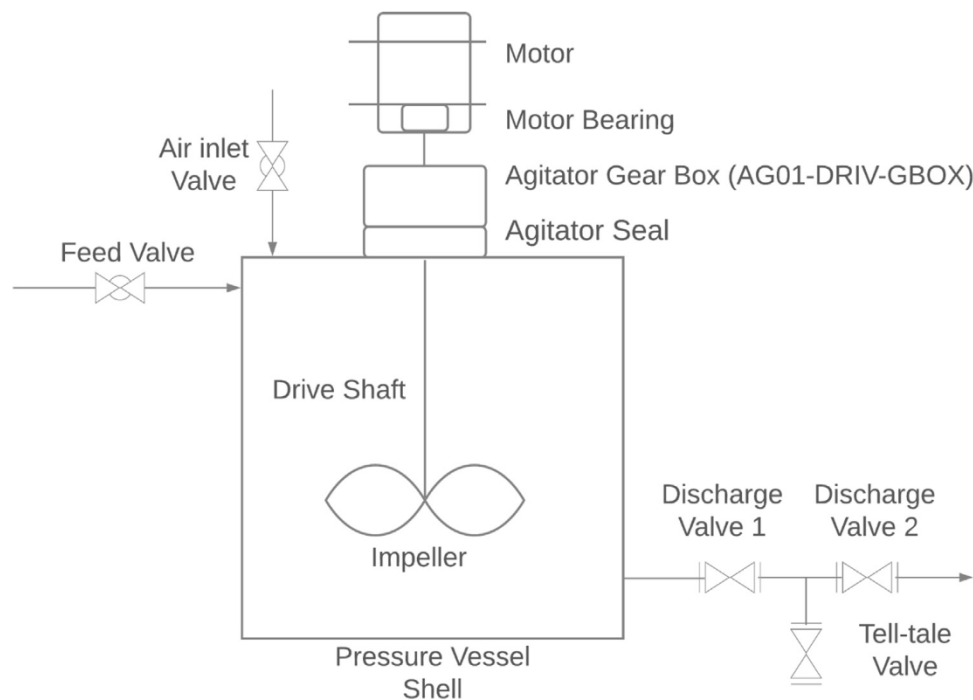


Fig. 9. Overview of key components in the pressure vessel considered in the FMEA process.

Explanation for: component_1000.10.10-RV001-AG01-DRIV-MOTR Type 'Object in Fault State'

```
component_1000.10.10-RV001-AG01-DRIV-MOTR hasMalfunction malfunction_due_to_Earth_Isolation_fault_1000.10.10-RV001-AG01-DRIV-MOTR
hasMalfunction some (realizedIn some owl:Thing) SubClassOf 'Object in Fault State'
malfunction_due_to_Earth_Isolation_fault_1000.10.10-RV001-AG01-DRIV-MOTR realizedIn failure_event
```

Fig. 10. Inference that component_1000.10.10-RV001-AG01-DRIV-MOTR is 'ObjectInFaultState' an due to earth isolation fault (replicates Fig. 7).

failure mode codes are drawn from ISO 14224 (ISO 14224, 2016), which is widely used in the petrochemical process industry. The two columns for failure mechanism and its superclass failure mechanism category are also aligned to ISO 14224 data requirements.

In the ontology, the *Failure Mechanism* column in Table 4 is represented as *malfunction* individuals. The wording used in these fields is representative of the language used by the industry partner. The only fields we have added to this are (a) one for a *FMEA_ID*, which allows us to uniquely identify each row of the table, and (b) a field to indicate if the item is functionally critical or not to the operation of the pressure vessel. If marked as critical then failure of this item will shut down an operating pressure vessel. Most of the items in this pressure vessel system are critical for an operating unit to continue to function. However, there is a small number of items that are necessary for maintenance but not for operation. One of these is the tell-tale valve. This valve is installed between two block valves as a means of determining if the block valves, which are only used when the vessel is shut down, are leaking. A suspected failure of the tell-tale valve, or any of its components, will not result in immediate shut down of the pressure vessel, and we wish to demonstrate this with the ontology.

We imported instance data from this FMEA table (extract shown in Fig. 4) into our ontology using the Owlready2 Python package (Lamy, 2017). Note that this is not the only data extraction method available and other tools including Karma (Shubham Gupta et al., 2012) and tabOTTR (Skjæveland et al., 2018) are also suitable for this task. The import script first loads the four ontology modules and arranges them according to the import structure shown in Fig. 2. The script then reads data using the pandas (McKinney et al., 2010) Python library. This data includes the FMEA table and a table containing the system and subsystem breakdown for the compo-

nents (drawn from the diagram provided by our industrial partner). The script iterates through the rows in the FMEA table, creates relevant individuals and relationships, and adds component individuals to the FSO ontology module and malfunction individuals to the FMO module. Note that that in this example, we automate the data import process. Therefore, no new classes have been created in the ASO to categorize the parts of the system (such as the Heaters class in Fig. 7). Instead, all assets in this example are of type 'Component' from the FSO. For this reason, we have not added all individuals directly into the ASO OWL file, as was done in the example in Section 5. The Python script used for this data import is available in the GitHub repository provided in Section 1.

Fig. 10 shows a malfunction experienced by a component – an earth fault on the agitator drive motor *component_1000.10.10-RV001-AG01-DRIV-MOTR* which results in the drive motor being in a fault state. Most of the components in this system are considered critical, such that, if the component fails then the system fails. This is demonstrated in Fig. 11: failure of the agitator drive motor system propagates to render the top level (the pressure vessel) *component_1000.10.10-RV001* an *Object in Fault State*.

We also explore some more complex cause and effect reasoning, associated with non-critical failures. For example, if the discharge block valve or the tell-tale valve is failed while the PV is in operation (called a hidden failure), the PV will not shut down, provided that the valve is already in the open position. Fig. 12 shows that the PV discharge block valve *component_1000.10.10-RV001-IT02* has a number of subcomponents (valve body, actuator, controller, etc.), one of which is *component_1000.10.10-RV001-IT02-XV02764*. Fig. 13 shows that failure of the valve body will propagate to give a failure of the discharge block valve. However, it will not propagate to a

Explanation for: component_1000.10.10-RV001 Type 'Object in Fault State'

```
component_1000.10.10-RV001 hasFunctionalPart component_1000.10.10-RV001-AG01
component_1000.10.10-RV001-AG01 hasFunctionalPart component_1000.10.10-RV001-AG01-DRIV-MOTR
hasMalfunction some (realizedIn some owl:Thing) SubClassOf 'Object in Fault State'
component_1000.10.10-RV001-AG01-DRIV-MOTR hasMalfunction malfunction_due_to_Earth_Isolation_fault_1000.10.10-RV001-AG01-DRIV-MOTR
hasFunctionalPart some 'Object in Fault State' SubClassOf 'Object in Fault State'
malfunction_due_to_Earth_Isolation_fault_1000.10.10-RV001-AG01-DRIV-MOTR realizedIn failure_event
```

Fig. 11. Inference that component_1000.10.10-RV001-AG01 is 'ObjectInFaultState' an due to earth isolation fault (replicates Fig. 8).

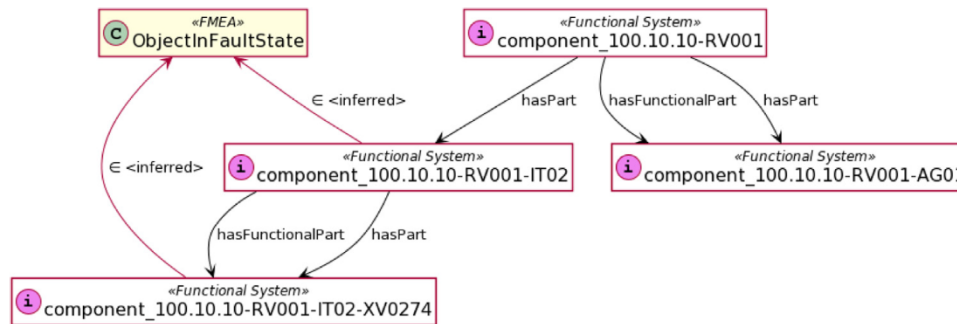


Fig. 12. Schematic illustrating reasoning on components for the pressure vessel test use case.

Explanation for: component_1000.10.10-RV001-IT02 Type 'Object in Fault State'

```
component_1000.10.10-RV001-IT02 hasFunctionalPart component_1000.10.10-RV001-IT02-XV02764
component_1000.10.10-RV001-IT02-XV02764 hasMalfunction malfunction_due_to_Clearance_alignment_failure_1000.10.10-RV001-IT02-XV02764
hasMalfunction some (realizedIn some owl:Thing) SubClassOf 'Object in Fault State'
malfunction_due_to_Clearance_alignment_failure_1000.10.10-RV001-IT02-XV02764 realizedIn failure_event
hasFunctionalPart some 'Object in Fault State' SubClassOf 'Object in Fault State'
```

Fig. 13. Inference that component_1000.10.10-RV001-IT02 is an 'ObjectInFaultState' (demonstration of Fig. 12).

failure of the PV. The same is true for the tell-tale valve. This is consistent with the engineering output we seek for this situation.

To perform this reasoning, no additional classes are required in the ontology. Instead, reasoning is supported by adding an additional component hierarchy to the ontology. So far, we have described a *functional component hierarchy* where components are arranged using the 'hasFunctionalPart' relation (from the upper ontology). This hierarchy describes the system's functional dependencies. The additional hierarchy describes the *physical* component hierarchy and uses the weaker 'hasPart' relation. If a component is an 'ObjectInFaultState', the reasoner will infer that any functional parent is also an 'ObjectInFaultState' (as demonstrated in Section 5). However, a parent in a *physical* component hierarchy will not necessarily be an 'ObjectInFaultState'.

7. Discussion and future work

This paper demonstrates an ontology-based approach to make the classes and relationships held in spreadsheet tables explicit. Using FMEA as a case study, we demonstrate the ability to describe a system hierarchy, supporting automated reasoning to test and infer dependencies. Final failure effects at the system level are propagated from component failures using OWL-DL reasoning. The contribution is a practical example of the use of ontologies on a real industry problem.

Having built the ontology modules, one each for the asset hierarchy and the functional system, and another for semi-structured data outputs from a FMEA process described in the IEC 60812 FMEA standard, we test the ontology on instance data from the same standard. We then import instance data from a real industry use case and confirm that the same reasoning process, to infer cause

and effect relationships, works as expected. This ability to reuse ontologies is a key but seldom demonstrated value of ontologies.

When we distinguish, as in the pressure vessel use case, between items that are critical and non-critical for operations, we can explore the different relationships in complex engineering systems. An item is deemed critical if a functional failure of the item has an impact on the top level of the hierarchy, in this case the pressure vessel itself. However a failure of a non-critical item, for example the actuator on the tell-tale valve, results in failure of the tell-tale valve but does not lead to failure of the pressure vessel. These sorts of checks on how failures propagate through the equipment hierarchy are time-consuming for engineers to perform manually, and there is opportunity to improve quality control of FMEA processes by implementing these checks using ontologies. Eventually, we see the opportunity for instance data on asset hierarchical relationships to be ingested directly from drawings that have appropriate digital uniform resource identifier (URI) tags and URI relationships, informed by a standardized classification scheme defined to reflect the function of the object, such as the scheme set out in IEC 81346:2019 (IEC, 2019).

After decades of trying to develop ontologies in sector and organisational silos, the engineering community has recognized a need to collaborate, co-create, and share ontologies in order to move forward. Examples include (1) ongoing work by the Norwegian oil and gas sector to produce ontologically consistent Material Master data based on ISO 15926-14, for which parts of the ontology are public (<https://github.com/Sirius-sfi/aibel-mmd-ontology>), and (2) the formation of the global Industrial Ontologies Foundry (IOF) consortium and their work-in-progress open-source set of axioms and definitions for an industrial ontology. Both of these developments adhere to the principles of the BFO top level ontology (Smith et al., 2019) and hence reduce the effort of onto-

logical development and maintenance. The modular FMEA and asset ontologies presented here are aligned to ISO 15926-14 but could readily be adapted for BFO 2020. Much engineering work is governed by ISO, IEC and other standards bodies. So, the move to formalize upper ontologies as ISO/IEC standards is a positive step forward for those in the engineering community working to ensure standardized, reusable, interoperable modular ontologies for industry.

Future work to further develop the ontology modules and their ability to perform complex reasoning will include testing on a wider range of industrial use case FMEAs and exploring the potential to identify common cause failures (CCFs). CCFs occur on multiple items, which would otherwise be considered independent, and result from a single cause (IEC, 2018). This is a particularly challenging area of engineering design practice, especially for safety critical equipment (Rausand and Høyland, 2003).

Declaration of Competing Interest

The authors report no declarations of interest.

References

- Dittman, L., Rademacher, T., Zelewski, S., 2004]. Performing FMEA using ontologies. *Evanston, USA In: The International Workshop on Qualitative Reasoning*, 18, pp. 209–216.
- Ebrahimipour, V., Rezaie, K., Shokravi, S., 2010]. An ontology approach to support FMEA studies. *Expert Syst. Appl.* 37 (1), 671–677.
- Hodkiewicz, M., Klüwer, J.W., Woods, C., Smoker, T., French, T., 2020. Digitalization and reasoning over engineering textual data stored in spreadsheet tables. *IFAC-PapersOnLine* 53 (3), 239–244.
- Horridge, M., 2011]. Justification Based Explanation in Ontologies. The University of Manchester, PhD Thesis.
- Horridge, M., Parsia, B., Sattler, U., 2008]. Explanation of OWL entailments in Protégé 4. *Proceedings of the 2007 International Conference on Posters and Demonstrations*, 60–61.
- IEC 60812, 2006a]. Analysis Techniques for System Reliability-Procedure for Failure Mode and Effects Analysis (FMEA), Technical Report.
- IEC 60812, 2006b]. Analysis Techniques for System Reliability-Procedure for Failure Mode and Effects Analysis (FMEA), Technical Report.
- IEC, 2018]. IEC 60812 Failure Modes and Effects Analysis (FMEA and FMECA). IEC, Geneva, Switzerland, Technical Report.
- IEC, 2019]. IEC 81346-2 Industrial Systems, Installations and Equipment and Industrial Products – Structuring Principles and Reference Designations – Part 2: Classification of Objects and Codes for Classes. IEC, Geneva, Switzerland, Technical Report.
- ISO 14224, 2016]. Petroleum, Petrochemical and Natural Gas Industries – Collection and Exchange of Reliability and Maintenance Data for Equipment. ISO, Technical Report.
- ISO 15926-14, 2019]. Industrial Automation Systems and Integration – Integration of Life-Cycle Data for Process Plants Including Oil and Gas Production Facilities – Part 14: Data Model Adopted for OWL 2 Direct Semantics. Community Draft. ISO.
- Laaroussi, A., Fiès, B., Vankeisbelck, R., Hans, J., 2007]. Ontology-aided FMEA for construction products. In: *Bringing ITC Knowledge to Work*, 24th W78 Conference, vol. 26(6), Maribor, pp. 189–194.
- Lamy, J.-B., 2017]. Owlready: ontology-oriented programming in Python with automatic classification and high level constructs for biomedical ontologies. *Artif. Intell. Med.* 80, 11–28.
- McKinney, W., et al., 2010]. Data structures for statistical computing in Python. In: *Proceedings of the 9th Python in Science Conference*, vol. 445, Austin, TX, pp. 51–56.
- Molhanec, M., Zhuravskaya, O., Povolotskaya, E., Tarba, L., 2011]. The ontology based FMEA of lead free soldering process. *Proceedings of the 2010 33rd International Spring Seminar on Electronics Technology (ISSE)*, 360–364.
- Rausand, M., Høyland, A., 2003]. *System Reliability Theory: Models, Statistical Methods, and Applications*. John Wiley & Sons.
- Rehman, Z., Kifor, C.V., 2016]. An ontology to support semantic management of FMEA knowledge. *Int. J. Comput. Commun. Control* 11 (4), 507–521.
- Gupta, S., Szekely, P., Knoblock, C.A., Goel, A., Taheriyani, M., Muslea, M., 2012]. Karma: a system for mapping structured sources into the semantic web. *Extended Semantic Web Conference*, 430–434.
- Skjæveland, M.G., Lupp, D.P., Karlsen, L.H., Forssell, H., 2018]. Practical ontology pattern instantiation, discovery, and maintenance with reasonable ontology templates. *International Semantic Web Conference*, 477–494.
- Smith, B., Ameri, F., Cheong, H., Kiritis, D., Sormaz, D., Will, C., Otte, J.N., 2019]. A first-order logic formalization of the industrial ontology foundry signature using basic formal ontology. In: *Proceedings of the Joint Ontology Workshops (JOWO)*, Graz.
- Staab, S., Studer, R., 2010. *Handbook on Ontologies*. Springer, Science & Business Media.
- Studer, R., Benjamins, V.R., Fensel, D., 1998]. Knowledge engineering: principles and methods. *Data Knowl. Eng.* 25 (1–2), 161–197.
- Uschold, M., Gruninger, M., 1996]. *Ontologies: principles, methods and applications*. *Knowl. Eng. Rev.* 11, 93–136.
- Xiuxu, Z., Yuming, Z., 2012]. Application research of ontology-enabled process FMEA knowledge management method. *Int. J. Intell. Syst. Appl.*, 34–40.